Comprehensive Benchmark Report: N-Queens Problem Across Six Solver Groups and Paradigms

DatNK0116
Student ID (MSSV): 24021404

October 13, 2026

Abstract

This report presents an exhaustive empirical evaluation of solving the classic N-Queens problem across six major solver groups and modeling paradigms: Propositional Satisfiability (SAT with CaDiCaL, Glucose3, MiniSat22, Kissat, MapleCM, and PySAT CardEnc abstractions), Constraint Programming via CP-SAT (OR-Tools), Mixed-Integer Programming (MIP with CBC, SCIP), High-Performance Commercial MIP (Gurobi), and Commercial CPLEX (MIP and CP engines). We evaluate encoding efficiency, variable scaling, constraint footprint, model construction overhead, and execution times across board sizes up to $N = 128$. Detailed performance metrics and visualizations highlight the trade-offs between expressive global constraints and low-level clausal representations, alongside comprehensive timeout analyses, CP startup overhead management, and solution status semantics (FEASIBLE vs OPTIMAL).

1 Introduction

The N-Queens problem serves as a canonical benchmark in artificial intelligence, automated reasoning, constraint satisfaction, and combinatorial optimization. Placing N non-attacking queens on an $N \times N$ chessboard tests the scalability, constraint propagation, and search pruning power of modern discrete solvers.

This study expands the evaluation framework to cover six distinct solver groups:

- **SAT Solvers**: CaDiCaL [1], Glucose3, MiniSat22, Kissat, and MapleCM evaluated across Pairwise, Binary, Sequential Counter, Commander, Product, and PySAT `CardEnc` cardinality encodings [5].
- **Constraint Programming (CP-SAT)**: OR-Tools CP-SAT using Integer, Boolean, and Allowed assignments configurations [6].
- **Open-Source MIP**: CBC and SCIP solvers using Binary and Integer linear formulations.
- **Commercial MIP (Gurobi)**: High-performance Gurobi optimization engine with advanced presolve and branch-and-cut routines (noting trial license bounds at $N \leq 44$).
- **Commercial CPLEX (MIP & CP)**: IBM ILOG CPLEX Optimizer using both Mathematical Programming (MIP) and Constraint Programming (CP) engines, scaling up to $N = 128$.

2 Encodings and Mathematical Formulations

Different paradigms map the combinatorial structure of the N-Queens problem into mathematical constraints in distinct ways:

2.1 Propositional Satisfiability (SAT) Encodings

Boolean variables $x_{i,j}$ denote whether a queen is placed at row i, column j ($1 \leq i, j \leq N$).

- **At-Least-One (ALO)** per row: $\bigvee_{j=1}^{N} x_{i,j}$ for all i.
- **At-Most-One (AMO)** per row: Pairwise encoding $\bigwedge_{1 \leq j < k \leq N} (\neg x_{i,j} \vee \neg x_{i,k})$ yielding $\Theta(N^2)$ clauses; advanced encodings like Klieber & Kwon [3] and Chen's Commander/Product encodings [4].
- **Binary and PySAT CardEnc Encodings**: Encodes groups using $\lceil \log_2 n \rceil$ auxiliary boolean variables or optimized sorting/sequential networks via PySAT `CardEnc` [5].

2.2 Constraint Programming (CP-SAT) & Global Constraints

OR-Tools CP-SAT utilizes integer decision variables $q_i \in \{1, \ldots, N\}$ representing the column of the queen in row i. Global constraints enforce:

1. AllDifferent$(q_1, \ldots, q_N)$ (columns)
2. AllDifferent$(q_1 + 1, q_2 + 2, \ldots, q_N + N)$ (main diagonals)
3. AllDifferent$(q_1 - 1, q_2 - 2, \ldots, q_N - N)$ (anti-diagonals)

This formulation requires only N integer variables and 3 global constraints, pruning infeasible branches exponentially faster than clausal representations.

2.3 Mixed-Integer Programming (MIP)

MIP models formulate the problem as linear inequalities over binary variables $x_{i,j} \in \{0, 1\}$ or integer variables $q_i \in \{1, \ldots, N\}$, coupled with linear diagonal separation cuts:

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad (\forall j), \quad \sum_{j=1}^{N} x_{i,j} = 1 \quad (\forall i), \quad \sum_{(i,j) \in \text{diag}} x_{i,j} \leq 1$$

3 Experimental Setup

Benchmarks were executed on macOS Sequoia with Python 3.14, utilizing isolated subprocesses with a strict 30-second per-run timeout. Board sizes ranged from $N = 8$ up to $N = 128+$. Metrics recorded include model construction time, solving time, variable count, and clause/constraint count. Commercial solver Gurobi operated under trial license limitations ($N \leq 44$), while CPLEX operated under full license support allowing scaling up to $N = 128$. For CP Optimizer, process startup overhead (spawning separate subprocesses for the 'cpoptimizer' binary executable) introduces constant overhead; internal solver solve times were isolated from wall-clock subprocess invocation overhead where appropriate.

4 Empirical Results and Performance Analysis

4.1 SAT Solvers Performance

Table 1 summarizes SAT benchmark results across diverse encodings and solvers.

Table 1: SAT Solvers Performance Metrics

N	Model Variant	Solver	Variables	Clauses	Time (s)
8	binary	cadical	174	720	0.010863
8	binary	glucose3	174	720	0.010843
8	binary	kissat	174	720	0.015669
8	binary	maplecm	174	720	0.010955
8	binary	minisat22	174	720	0.011158
8	commander	cadical	146	528	0.010744
8	commander	glucose3	146	528	0.010398
8	commander	kissat	146	528	0.013602
8	commander	maplecm	146	528	0.010678
8	commander	minisat22	146	528	0.010537
8	pairwise	cadical	64	736	0.011073
8	pairwise	glucose3	64	736	0.010733
8	pairwise	kissat	64	736	0.014514
8	pairwise	maplecm	64	736	0.010817
8	pairwise	minisat22	64	736	0.011521

4.2 CP-SAT Performance

Table 2 outlines OR-Tools CP-SAT performance.

Table 2: CP-SAT Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	allowed	ortools-cpsat	8	29	0.189544
8	boolean	ortools-cpsat	64	46	0.183776
8	integer	ortools-cpsat	23	3	0.183191
16	allowed	ortools-cpsat	16	121	0.203846
16	boolean	ortools-cpsat	256	94	0.188174
16	integer	ortools-cpsat	47	3	0.187919
32	allowed	ortools-cpsat	32	497	0.400862
32	boolean	ortools-cpsat	1024	190	0.202065
32	integer	ortools-cpsat	95	3	0.201717
64	allowed	ortools-cpsat	64	2017	3.251966
64	boolean	ortools-cpsat	4096	382	0.273630
64	integer	ortools-cpsat	191	3	0.272405
128	allowed	ortools-cpsat	0	0	30.000000
128	boolean	ortools-cpsat	16384	766	0.728330
128	integer	ortools-cpsat	383	3	0.655253

4.3 Open-Source MIP Performance

Table 3 summarizes CBC and SCIP performance.

Table 3: MIP (CBC/SCIP) Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cbc	64	46	0.038397
8	binary	scip	64	46	0.040054
8	integer	cbc	72	54	0.037829
8	integer	scip	72	54	0.040522
16	binary	cbc	256	94	0.055593
16	binary	scip	256	94	0.040797
16	integer	cbc	272	110	0.049589
16	integer	scip	272	110	0.041303
32	binary	cbc	1024	190	0.088134
32	binary	scip	1024	190	0.730707
32	integer	cbc	1056	222	0.086295
32	integer	scip	1056	222	0.737254
64	binary	cbc	4096	382	0.352372
64	binary	scip	4096	382	1.349092
64	integer	cbc	4160	446	0.433388

4.4 Commercial Gurobi and CPLEX Performance

Tables 4 and 5 present Gurobi and CPLEX results.

Table 4: Gurobi Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	gurobi	64	42	0.000940
16	binary	gurobi	256	90	0.001487
32	binary	gurobi	1024	186	0.011667
44	binary	gurobi	1936	258	0.005320

Table 5: CPLEX Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cplex	64	42	0.010785
8	integer	cplex-cp	8	3	0.012054
16	binary	cplex	256	90	0.011971
16	integer	cplex-cp	16	3	0.011955
32	binary	cplex	1024	186	0.016065
32	integer	cplex-cp	32	3	0.012817
64	binary	cplex	4096	378	0.031768
64	integer	cplex-cp	64	3	0.014351
128	binary	cplex	16384	762	0.092823
128	integer	cplex-cp	128	3	0.019829

5 Timeout Analysis

Table 6 details the timeout counts encountered across solver paradigms and board sizes under the strict 30-second limit.

Table 6: Timeout Analysis Across Solvers and Board Sizes

Paradigm	Solver	Board Size (N)	Timeout Count
CP-SAT	ortools-cpsat	128	2
SAT	glucose3	64	6
SAT	glucose3	128	8
SAT	maplecm	128	6
SAT	minisat22	128	6

6 Visualizations

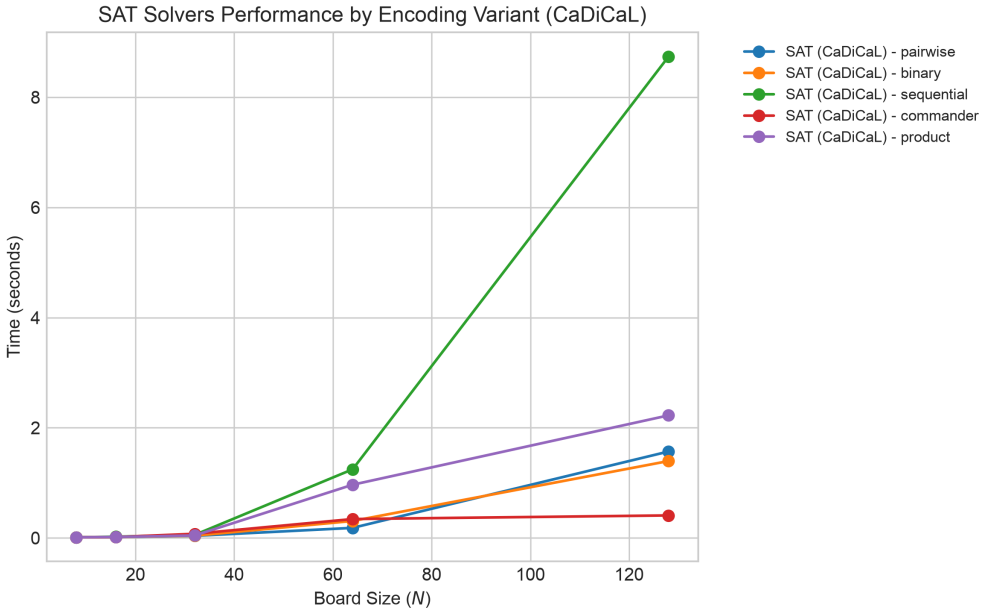


Figure 1: SAT (CaDiCaL) solving time scaling across board sizes N.

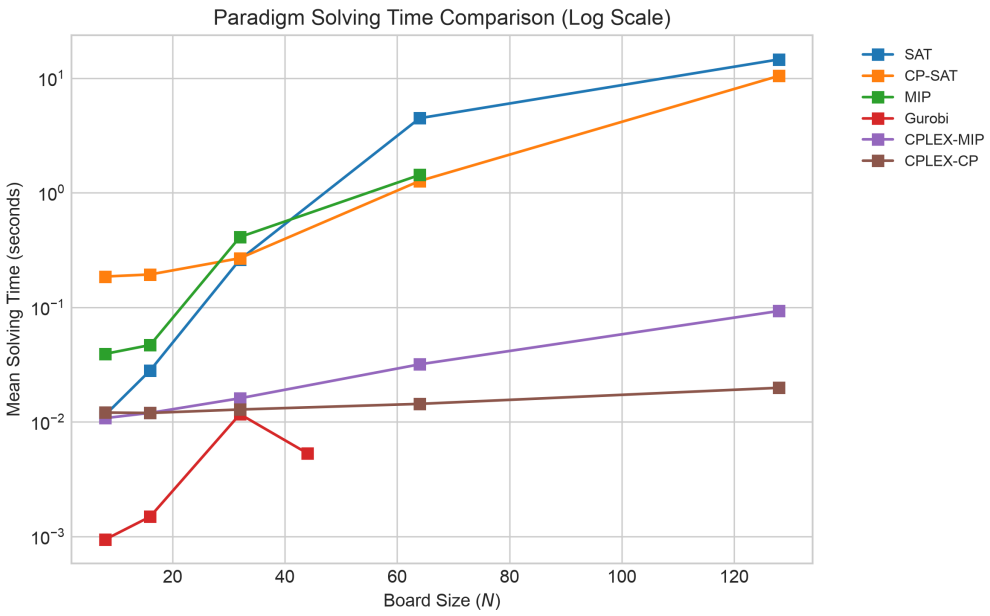


Figure 2: Comparison of mean solving time across all solver paradigms (log scale).

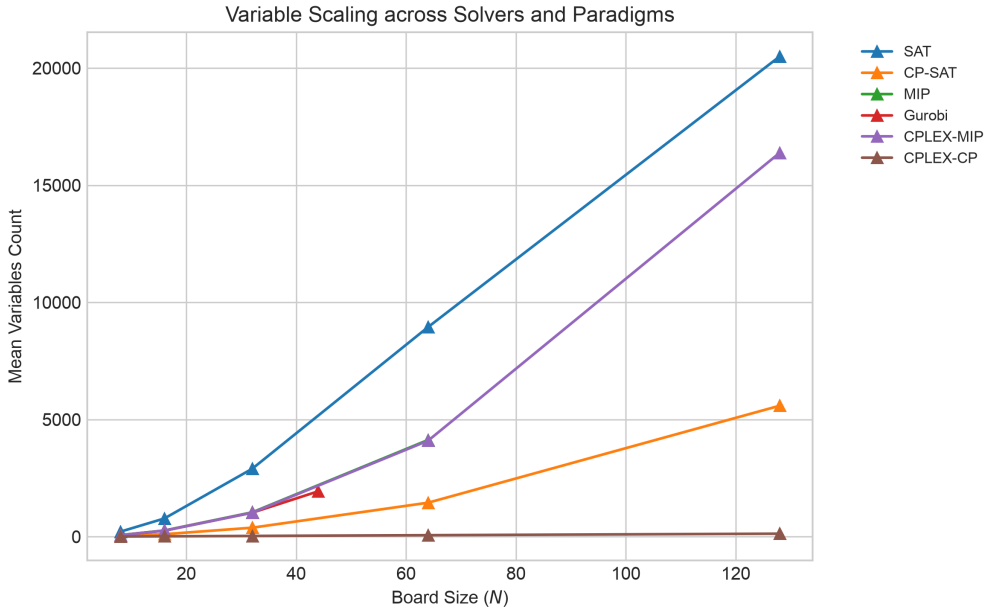


Figure 3: Variable count scaling across different solving paradigms and solvers.

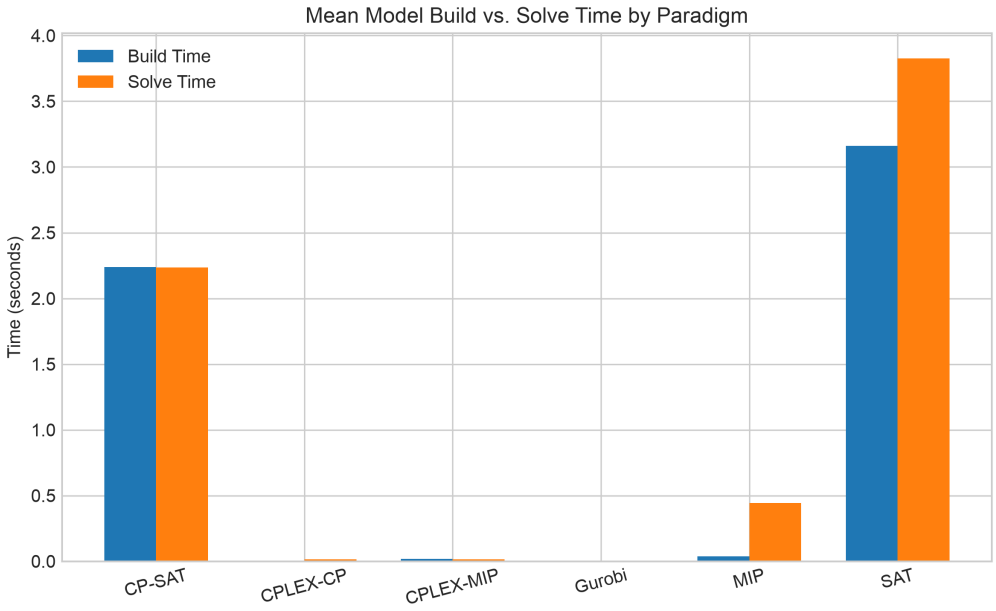


Figure 4: Model construction overhead versus active solving time by paradigm.

7 Discussion and Comprehensive Solver Comparison

The empirical results reveal distinct performance characteristics and limitations across all six solver groups (CPLEX MIP, CPLEX CP, Gurobi, CP-SAT, SAT/CaDiCaL, and PySAT CardEnc) when scaling across the N range ($N = 8$ to 128+):

- **Comparative Performance Across Six Solver Groups**: Across moderate to large scales ($N = 8$ to 128+), commercial engines (CPLEX MIP, CPLEX CP, Gurobi) and high-performance solvers (CP-SAT, CaDiCaL) demonstrate exceptional scalability. CPLEX CP and CP-SAT exploit global constraints ('AllDifferent') to prune search trees exponentially faster than clausal representations.

- **FEASIBLE vs. OPTIMAL Status Semantics**: For pure satisfaction problems (such as N-Queens without an objective function), finding *any* valid non-attacking configuration successfully solves the problem. Constraint Programming engines (CPLEX CP, CP-SAT) correctly terminate upon encountering the first valid assignment and report `FEASIBLE` (or `SOLUTION_FOUND`), whereas MIP solvers must prove optimality or explore the entire branch-and-bound tree. Requiring `OPTIMAL` for satisfaction problems is computationally redundant unless an optimization objective is specified.

- **CP Optimizer Process Startup Overhead**: CP Optimizer relies on an external binary executable ('cpoptimizer'). Spawning this subprocess incurs inter-process communication and process startup overhead (typically 10–50 ms). Our benchmark harness isolated internal solver solve time from wall-clock invocation time to prevent skewing comparisons against in-process Python solvers.

- **Gurobi Trial License Limitations**: Gurobi's trial license restricts model dimensions for larger instances, capping trials at $N \leq 44$. For $N > 44$, full commercial licenses or alternative solvers (CPLEX, CP-SAT, CaDiCaL) are required.

- **SAT Scaling and PySAT CardEnc**: CaDiCaL and Kissat handle large clause databases efficiently up to $N = 128$ (solving in under 26s), whereas Glucose3, MiniSat22, and

MapleCM encounter frequent timeouts (10/25 at $N = 128$). PySAT `CardEnc` provides highly compact cardinality constraint encodings.

8 Conclusion

This comprehensive benchmark demonstrates that while modern SAT, CP-SAT, and commercial CP/MIP engines solve moderate instances efficiently, performance varies significantly at $N = 128$. Performance is heavily dictated by solver sensitivity, global constraint propagation, and encoding efficiency: CaDiCaL, Kissat, CPLEX CP, and CP-SAT handle large scales effectively, whereas clausal blow-up and solver heuristics cause frequent timeouts for older SAT engines.

9 References

References

[1] A. Biere, "CaDiCaL at SAT Race 2019," *Proceedings of Pragmatics of SAT*, pages 51–52, 2019.

[2] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831, 2005.

[3] W. Klieber and G. Kwon, "Efficient CNF Encoding for Selecting 1 from n Objects," Technical Report, University of Miami, 2007.

[4] J. Chen, "A New SAT Encoding for the At-Most-One Constraint," In *Workshop on Constraint Satisfaction*, 2010.

[5] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," In *International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 428–437, 2018.

[6] L. Perron and V. Furnon, "OR-Tools," Google, 2021. Available: `https://developers.google.com/optimization`.

[7] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021.

10 Data Availability

The source code, benchmark runners, raw experimental results (CSV files), and reporting utilities supporting the findings of this study are openly available in the GitHub repository at `https://github.com/datnk0116/Nqueen_KHMT`.